

Vibe coding vs spec-driven development

Dimension	Vibe coding	Spec-driven development
Primary intent	Rapid exploration and quick generation	Controlled implementation from defined intent
Source of truth	Prompt, chat history, and developer memory	Versioned specification, plan, tests, and acceptance criteria
Best fit	Prototype, demo, learning exercise, internal experiment	Production system, regulated workload, multi-team delivery, long-lived product
Risk profile	Higher risk of drift, hidden assumptions, technical debt, and weak traceability	Lower risk through explicit constraints, reviews, validation, and auditability
Human role	Prompt, observe, correct, and iterate	Define intent, review specification, govern decisions, verify outputs

scope, assumptions, approval gates, and verification evidence explicit. It also allows work to be decomposed into smaller, auditable tasks, which is essential when agents are allowed to act across repositories, pipelines, test environments, issue trackers, and cloud platforms.

Spec-driven development versus vibe coding

Vibe coding and spec-driven development are not merely two tool preferences; they represent two different philosophies of working with AI. Vibe coding is exploratory, conversational, and fast. A developer describes what they want, receives generated code, tests or observes the outcome, adjusts the prompt, and iterates. It is useful when the goal is discovery, prototyping, learning a new framework, creating a throwaway proof of concept, or testing whether an idea has value. It lowers the entry barrier and gives teams a quick way to move from thought to working sample.

Vibe coding appears weak when the work must survive beyond the prototype. Chat history is not a reliable architecture record. Generated code may work locally but miss edge cases, non-functional requirements, security controls, naming conventions, observability requirements, or integration contracts. Different prompts can produce different implementations for the same feature, and later contributors may not understand why specific choices were made. As

the system grows, the early speed of vibe coding can be offset by rework, duplicated logic, inconsistent patterns, and fragile maintenance.

Spec-driven development takes a different route. It begins by making intent explicit before implementation begins. It asks teams to define the expected behaviour, constraints, data ownership, quality requirements, non-goals, compliance expectations, and verification approach. AI agents still accelerate implementation, but they operate inside a more carefully described boundary. The trade-off is clear: SDD requires more upfront thinking, but it improves maintainability, traceability, quality, and enterprise confidence. A practical enterprise strategy is not to reject vibe coding entirely. Teams can use vibe coding for early exploration, then convert the learning into a living specification before production-grade development begins.

Advantages of spec-driven development

The first advantage of SDD is clarity. A specification removes much of the guesswork that typically sits between a business idea and implementation. It defines the expected result in a way that product owners, architects, developers, testers, security teams, and AI agents can all interpret. This shared clarity reduces repeated clarification cycles and helps teams avoid building features that are technically impressive but misaligned with business intent.

The second advantage is traceability. In enterprise environments, teams must often prove why a change was made, which requirement it satisfies, what tests validate it, and who approved the release. SDD creates a natural chain from requirement to design, from design to task, from task to code, and from code to evidence. This is valuable not only for audits, but also for daily engineering discipline. When a defect appears, the team can examine whether the specification was incomplete, the implementation violated the specification, or the tests failed to cover the intended behaviour.

The third advantage is better governance of AI-generated code. AI coding agents are powerful but not inherently accountable. They can produce plausible code that compiles while still violating security principles, architectural standards, licensing expectations, privacy rules, or performance constraints. A strong specification acts as a guardrail. It states what the AI system may assume, what it must not change, what approval is required, what data may be accessed, and what evidence is needed before the output is accepted.

The fourth advantage is maintainability. A codebase generated from vague prompts may function in the short term, but future teams will struggle to understand its assumptions. SDD keeps the rationale visible. It supports onboarding, refactoring, modernisation, and impact analysis because the design intent is available